🍕 Number of Ways of Cutting a Matrix / Pizza

> **Pattern:** 2D Grid + Recursive Partitioning + Memoization + 2D Prefix Sum + 3D DP **Difficulty:** Hard
> **Core Learning:** How to build a solution step-by-step instead of jumping directly to DP.

---

1. Problem Statement

Given a matrix containing `0`s and `1`s and an integer `k`, divide the matrix into exactly `k` pieces.

Each final piece must contain **at least one** `1`.

A cut can be:

Horizontal

```
Before:

A A A
A A A
B B B
B B B
```

The top part is given away.

The bottom part can be cut further.

Vertical

```
Before:

A A | B B
A A | B B
A A | B B
```

The left part is given away.

The right part can be cut further.

---

2. Example

```
matrix =

1 0 0
1 1 1
0 0 0

k = 3
```

We need:

```
3 pieces
```

Therefore:

```
number of cuts = k - 1 = 2
```

There are 3 valid ways.

---

3. First Important Insight: Do Not Jump to DP

Initially, it is tempting to think:

```
"Number of ways"
        ↓
      DP
```

But this is not enough.

The correct process is:

```
Problem
   ↓
What decisions can I make?
   ↓
What happens after one decision?
   ↓
What smaller problem remains?
   ↓
Write recursion
   ↓
Find repeated states
   ↓
```

```
Memoization
   ↓
Tabulation
   ↓
Optimize expensive operations
```

---

4. Start With One Cut

Suppose the current matrix is:

```
1 0 0
1 1 1
0 0 0
```

We can make a horizontal cut:

```
1 0 0  ← given away
---------
1 1 1
0 0 0  ← continue with this part
```

The remaining problem is now:

```
1 1 1
0 0 0
```

This is recursion.

The original problem:

```
Entire matrix
```

becomes:

```
Smaller remaining matrix
```

after one decision.

---

5. How Do We Represent the Remaining Matrix?

Initially:

```
1 0 0
1 1 1
0 0 0
```

Coordinates:

```
        col 0   col 1   col 2
      ┌──────────────────────
row 0 │  1       0       0
row 1 │  1       1       1
row 2 │  0       0       0
```

Initially, the current rectangle is:

```
start = (0, 0)
end   = (2, 2)
```

After horizontal cut after row `0`:

```
1 0 0  ← given away
---------
1 1 1
0 0 0  ← remaining
```

The remaining rectangle starts at:

```
row = 1
col = 0
```

So:

```
new start = (1, 0)
```

The bottom-right corner is still:

```
(2, 2)
```

Therefore:

```
(1, 0) → (2, 2)
```

---

6. Why Do We Only Store `(top, left)`?

The current rectangle always extends to the original bottom-right corner.

Initially:

```
(0, 0) → (rows - 1, cols - 1)
```

After a horizontal cut:

```
(r + 1, left) → (rows - 1, cols - 1)
```

After a vertical cut:

```
(top, c + 1) → (rows - 1, cols - 1)
```

The bottom boundary never changes.

The right boundary never changes.

Therefore, we only need:

```
top
left
```

plus:

```
cuts remaining
```

So the recursive state becomes:

```
solve(top, left, cuts)
```

Meaning:

> Count the number of ways to make `cuts` more cuts in the rectangle starting at `(top, left)` and extending to the original bottom-right corner.

---

7. State Transitions

Horizontal Cut

Suppose:

```
solve(top, left, cuts)
```

We cut after row `r`.

```
Rows top ... r
```

are given away.

The remaining part starts at:

```
r + 1
```

Therefore:

```
solve(r + 1, left, cuts - 1)
```

Rule

```
Horizontal cut:
    top changes
    left stays same
    cuts decreases by 1
```

Vertical Cut

Suppose we cut after column `c`.

```
Columns left ... c
```

are given away.

The remaining part starts at:

```
c + 1
```

Therefore:

```
solve(top, c + 1, cuts - 1)
```

Rule

```
Vertical cut:
    top stays same
    left changes
    cuts decreases by 1
```

---

8. Verify the State With Examples

Example 1

```
solve(0, 0, 2)
```

Horizontal cut after row `0`.

```
new top = 0 + 1 = 1
new left = 0
new cuts = 2 - 1 = 1
```

Therefore:

```
solve(1, 0, 1)
```

Example 2

```
solve(1, 0, 1)
```

Horizontal cut after row `1`.

```
new top = 1 + 1 = 2
new left = 0
new cuts = 1 - 1 = 0
```

Therefore:

```
solve(2, 0, 0)
```

Example 3

```
solve(1, 0, 1)
```

Vertical cut after column `0`.

```
new top = 1
new left = 0 + 1 = 1
new cuts = 1 - 1 = 0
```

Therefore:

```
solve(1, 1, 0)
```

9. Why Do We Need `k - 1` Cuts?

To create `k` pieces:

```
1 cut → 2 pieces
2 cuts → 3 pieces
```

```
3 cuts → 4 pieces
```

Therefore:

```
number of cuts = k - 1
```

So the initial call is:

```
solve(0, 0, k - 1)
```

For:

```
k = 3
```

we call:

```
solve(0, 0, 2)
```

---

10. The Important Rule: Every Given-Away Piece Must Have a 1

Suppose we make a horizontal cut:

```
1 0 0  ← given away
---------
1 1 1
0 0 0  ← remaining
```

The given-away part:

```
1 0 0
```

contains a 1.

Therefore:

```
valid cut
```

We can continue recursively.

Suppose:

```
0 0 0  ← given away
---------
1 1 1
```

The given-away part has no `1`.

Therefore:

```
invalid cut
```

We must not recurse.

So every transition is:

```
Try cut
   ↓
Does the piece being given away contain a 1?
   ↓
No → skip this cut
Yes → recurse on remaining piece
```

---

11. Base Case

Suppose:

```
cuts == 0
```

This means:

> No more cuts are allowed.

The remaining rectangle is the final piece.

Therefore:

```
Does the remaining rectangle contain a 1?
```

If yes:

```
return 1
```

If no:

```
return 0
```

So:

```
if cuts == 0:
    return 1 if current_rectangle_has_one else 0
```

---

12. Build the Brute-Force Recursion

Before writing code, write the logic:

```
solve(top, left, cuts):

    if no cuts remain:
        check final rectangle
        return 1 or 0

    ans = 0

    try every horizontal cut:

        if given-away top part contains 1:
            recurse on bottom part

    try every vertical cut:

        if given-away left part contains 1:
            recurse on right part

    return ans
```

---

13. Brute-Force `has_one()` Function

Initially, we can simply scan the rectangle.

```
def has_one(r1, c1, r2, c2):

    for i in range(r1, r2 + 1):

        for j in range(c1, c2 + 1):

            if matrix[i][j] == 1:
                return True

    return False
```

This means:

```
Check every cell from:

(r1, c1)

to:

(r2, c2)
```

If any cell is `1`:

```
return True
```

Otherwise:

```
return False
```

---

14. Full Brute-Force Recursive Code

```
def number_of_ways(matrix, k):

    rows = len(matrix)
    cols = len(matrix[0])
```

```
        def has_one(r1, c1, r2, c2):

            for i in range(r1, r2 + 1):

                for j in range(c1, c2 + 1):

                    if matrix[i][j] == 1:
                        return True

            return False

        def solve(top, left, cuts):

            # No more cuts.
            # The remaining rectangle is the final piece.
            if cuts == 0:

                if has_one(
                    top,
                    left,
                    rows - 1,
                    cols - 1
                ):
                    return 1

                return 0

            ans = 0

            # --------------------------------
            # Try horizontal cuts
            # --------------------------------
            for r in range(top, rows - 1):

                # Given-away top part:
                #
                # rows top ... r
                #
                # Must contain at least one 1.

                if has_one(
                    top,
                    left,
                    r,
                    cols - 1
                ):

                    # Continue with bottom part.
                    ans += solve(
                        r + 1,
                        left,
                        cuts - 1
                    )
```

```
        # ------------------------------
        # Try vertical cuts
        # ------------------------------
        for c in range(left, cols - 1):

            # Given-away left part:
            #
            # columns left ... c
            #
            # Must contain at least one 1.

            if has_one(
                top,
                left,
                rows - 1,
                c
            ):

                # Continue with right part.
                ans += solve(
                    top,
                    c + 1,
                    cuts - 1
                )

        return ans

    return solve(0, 0, k - 1)
```

---

15. Understand the Horizontal Loop

```
for r in range(top, rows - 1):
```

Suppose:

```
rows = 3
top = 0
```

Then:

```
r = 0
r = 1
```

These represent:

```
cut after row 0
cut after row 1
```

We cannot cut after the final row because then no bottom part would remain.

Therefore:

```
range(top, rows - 1)
```

Horizontal Dry Run

Initial:

```
solve(0, 0, 2)
```

Possible horizontal cuts:

```
r = 0
r = 1
```

`r = 0`

```
1 0 0  ← given away
---------
1 1 1
0 0 0  ← remaining
```

Next state:

```
solve(1, 0, 1)
```

`r = 1`

```
1 0 0
1 1 1  ← given away
---------
0 0 0  ← remaining
```

Next state:

```
solve(2, 0, 1)
```

---

16. Understand the Vertical Loop

```
for c in range(left, cols - 1):
```

Suppose:

```
cols = 3
left = 0
```

Then:

```
c = 0
c = 1
```

These represent:

```
cut after column 0
cut after column 1
```

Vertical Dry Run

Initial:

```
1 0 0
1 1 1
0 0 0
```

Cut after column `0`:

```
1 | 0 0
1 | 1 1
```

```
0 | 0 0
```

Given-away part:

```
1
1
0
```

Remaining part:

```
0 0
1 1
0 0
```

Next state:

```
solve(0, 1, cuts - 1)
```

---

17. Why Does the Recursive Call Return?

Suppose:

```
solve(1, 0, 1)
```

Horizontal cut after row `1`.

Next:

```
solve(2, 0, 0)
```

Now:

```
cuts == 0
```

So the base case executes.

The recursive call returns:

```
0 or 1
```

Then execution returns to:

```
solve(1, 0, 1)
```

The parent function then continues.

Important:

```
Recursive call returning
```

does not necessarily mean:

```
Entire function ends
```

It only means:

```
That particular branch finished.
```

Then the parent may try other cuts.

---

18. Problem With Brute Force

The `has_one()` function scans the rectangle every time.

For example:

```
has_one(...)
```

may scan:

```
100 cells
```

Then another cut may scan:

```
80 cells
```

Then another:

```
60 cells
```

The same cells are repeatedly examined.

Also, the recursion may reach the same state multiple times.

So we have two optimization opportunities.

---

19. Optimization 1: Memoization

Our state is:

```
solve(top, left, cuts)
```

Therefore:

```
memo[top][left][cuts]
```

can store the answer.

If we calculate:

```
solve(1, 0, 1)
```

once, and later reach:

```
solve(1, 0, 1)
```

again, we already know its answer.

So:

```
if state in memo:
```

```
    return memo[state]
```

20. Memoized Code

```
def number_of_ways(matrix, k):

    MOD = 10**9 + 7

    rows = len(matrix)
    cols = len(matrix[0])

    memo = {}

    def has_one(r1, c1, r2, c2):

        for i in range(r1, r2 + 1):

            for j in range(c1, c2 + 1):

                if matrix[i][j] == 1:
                    return True

        return False

    def solve(top, left, cuts):

        if cuts == 0:

            return 1 if has_one(
                top,
                left,
                rows - 1,
                cols - 1
            ) else 0

        state = (top, left, cuts)

        if state in memo:
            return memo[state]

        ans = 0

        # Horizontal cuts
        for r in range(top, rows - 1):

            if has_one(
                top,
                left,
                r,
```

```
                    cols - 1
                ):

                    ans += solve(
                        r + 1,
                        left,
                        cuts - 1
                    )

            # Vertical cuts
            for c in range(left, cols - 1):

                if has_one(
                    top,
                    left,
                    rows - 1,
                    c
                ):

                    ans += solve(
                        top,
                        c + 1,
                        cuts - 1
                    )

            memo[state] = ans % MOD

            return memo[state]

        return solve(0, 0, k - 1)
```

---

21. Optimization 2: Improve `has_one()`

Currently:

```
has_one(r1, c1, r2, c2)
```

scans the entire rectangle.

We want:

```
Can I know how many 1s are inside a rectangle in O(1)?
```

This is exactly what a:

2D Prefix Sum

does.

---

22. What Is a 2D Prefix Sum?

For a 1D array:

```
arr = [1, 2, 3, 4]
```

Prefix sum:

```
prefix = [1, 3, 6, 10]
```

Meaning:

```
prefix[i]
```

stores the sum from the beginning until index `i`.

For a matrix:

```
1 0 0
1 1 1
0 0 0
```

we create a 2D prefix table.

We use an extra row and extra column:

```
0 0 0 0
0
0
0
```

Why?

It makes boundary calculations easier.

---

23. 2D Prefix Sum Meaning

Define:

```
prefix[i][j]
```

as:

> The number of `1`s in the rectangle from `(0,0)` to `(i-1,j-1)`.

The indices are shifted because of the extra row and column.

For:

```
matrix =
1 0 0
1 1 1
0 0 0
```

The prefix table becomes:

```
      0  1  2  3
   ┌─────────────
0  │ 0  0  0  0
1  │ 0  1  1  1
2  │ 0  2  3  4
3  │ 0  2  3  4
```

For example:

```
prefix[2][2] = 3
```

means:

```
matrix rows 0..1
matrix cols 0..1
```

contains:

```
1 0
1 1
```

Total:

```
3 ones
```

---

24. 2D Prefix Sum Formula

For:

```
prefix[i][j]
```

the corresponding matrix cell is:

```
matrix[i - 1][j - 1]
```

Formula:

```
prefix[i][j] = (
    matrix[i - 1][j - 1]
    + prefix[i - 1][j]
    + prefix[i][j - 1]
    - prefix[i - 1][j - 1]
)
```

Why?

We combine:

```
top rectangle
+
left rectangle
+
current cell
```

But the top-left rectangle is counted twice.

Therefore:

```
subtract top-left once
```

---

25. Prefix Sum Formula Visualized

Suppose:

```
prefix[i][j]
```

We want:

```
┌──────────────┐
│ A │          │
├───┼──────────┤
│ B │ C        │
└───┴──────────┘
```

The formula:

```
A + B + C
```

double-counts the overlapping top-left region.

So:

```
prefix[i][j]
=
top
+
left
-
overlap
+
current cell
```

Therefore:

```
prefix[i][j] = (
    prefix[i-1][j]
    + prefix[i][j-1]
    - prefix[i-1][j-1]
    + matrix[i-1][j-1]
)
```

---

26. Rectangle Sum Query

Suppose we want:

```
rectangle from:

(r1, c1)

to:

(r2, c2)
```

The number of 1s is:

```
rectangle_sum = (
    prefix[r2 + 1][c2 + 1]
    - prefix[r1][c2 + 1]
    - prefix[r2 + 1][c1]
    + prefix[r1][c1]
)
```

---

27. Why This Formula Works

Imagine:

```
FULL PREFIX RECTANGLE
┌──────────────────────┐
│                      │
│      TARGET          │
│                      │
└──────────────────────┘
```

Start with the rectangle:

```
(0,0) → (r2,c2)
```

This includes too much.

Remove:

```
rows above r1
```

Remove:

```
columns before c1
```

But the top-left area was removed twice.

Add it back once.

Therefore:

```
target
=
full
-
top
-
left
+
overlap
```

This is the same inclusion-exclusion idea.

---

28. Example Rectangle Query

Matrix:

```
1 0 0
1 1 1
0 0 0
```

Suppose we want:

```
rows 1..2
cols 1..2
```

Target rectangle:

```
1 1
0 0
```

Number of ones:

```
1
```

Using prefix:

```
sum = (
    prefix[3][3]
    - prefix[1][3]
    - prefix[3][1]
    + prefix[1][1]
)
```

The answer is:

```
1
```

Instead of scanning the two-dimensional rectangle.

---

29. Why `rectangle_sum > 0` Means "Contains a 1"

The matrix contains only:

```
0
1
```

Therefore:

```
sum of rectangle
```

is simply:

```
number of 1s in that rectangle
```

Examples:

```
rectangle =

0 0
0 0

sum = 0
```

Therefore:

```
sum > 0 → False
```

No 1.

Another:

```
rectangle =

0 1
0 0

sum = 1
```

Therefore:

```
sum > 0 → True
```

Contains a 1.

Another:

```
rectangle =

1 1
1 0

sum = 3
```

Therefore:

```
sum > 0 → True
```

Contains at least one `1`.

So:

```
if rectangle_sum(...) > 0:
```

is exactly equivalent to:

```
if has_one(...):
```

but faster.

---

30. Optimized `has_one()`

Instead of:

```
def has_one(r1, c1, r2, c2):

    for i in range(r1, r2 + 1):

        for j in range(c1, c2 + 1):

            if matrix[i][j] == 1:
                return True

    return False
```

we do:

```
def has_one(r1, c1, r2, c2):

    return rectangle_sum(
        r1,
        c1,
        r2,
        c2
    ) > 0
```

Now:

```
Before:
O(area of rectangle)

After:
O(1)
```

---

31. Full Memoization + Prefix Sum Code

```
def number_of_ways(matrix, k):

    MOD = 10**9 + 7

    rows = len(matrix)
    cols = len(matrix[0])

    # ------------------------------------
    # Build 2D prefix sum
    # ------------------------------------

    prefix = [
        [0] * (cols + 1)
        for _ in range(rows + 1)
    ]

    for i in range(1, rows + 1):

        for j in range(1, cols + 1):

            prefix[i][j] = (
                matrix[i - 1][j - 1]
                + prefix[i - 1][j]
                + prefix[i][j - 1]
                - prefix[i - 1][j - 1]
            )
```

```
    # ------------------------------------
    # O(1) rectangle sum
    # ------------------------------------

    def rectangle_sum(r1, c1, r2, c2):

        return (
            prefix[r2 + 1][c2 + 1]
            - prefix[r1][c2 + 1]
            - prefix[r2 + 1][c1]
            + prefix[r1][c1]
        )

    memo = {}

    def solve(top, left, cuts):

        # Base case
        if cuts == 0:

            return 1 if rectangle_sum(
                top,
                left,
                rows - 1,
                cols - 1
            ) > 0 else 0

        state = (top, left, cuts)

        if state in memo:
            return memo[state]

        ans = 0

        # Horizontal cuts
        for r in range(top, rows - 1):

            if rectangle_sum(
                top,
                left,
                r,
                cols - 1
            ) > 0:

                ans += solve(
                    r + 1,
                    left,
                    cuts - 1
                )

        # Vertical cuts
        for c in range(left, cols - 1):

            if rectangle_sum(
```

```
                top,
                left,
                rows - 1,
                c
            ) > 0:

                ans += solve(
                    top,
                    c + 1,
                    cuts - 1
                )

        memo[state] = ans % MOD

        return memo[state]

    return solve(0, 0, k - 1)
```

---

32. Now Convert Recursion to Tabulation

Our recursive state is:

```
solve(top, left, cuts)
```

Therefore:

```
dp[top][left][cuts]
```

means:

> Number of ways to make `cuts` more cuts from the rectangle starting at `(top, left)`.

---

33. Recurrence

For horizontal cuts:

```
dp[top][left][cuts] += dp[r + 1][left][cuts - 1]
```

For vertical cuts:

```
dp[top][left][cuts] += dp[top][c + 1][cuts - 1]
```

Therefore:

```
dp[top][left][cuts]
```

depends on:

```
cuts - 1
```

So we calculate:

```
cuts = 0
cuts = 1
cuts = 2
...
cuts = k - 1
```

---

34. Base Case in Tabulation

Recursive:

```
if cuts == 0:

    return 1 if remaining rectangle has a 1 else 0
```

Tabulation:

```
for top:
    for left:

        if rectangle contains 1:

            dp[top][left][0] = 1
```

Otherwise:

```
dp[top][left][0] = 0
```

---

35. Tabulation Dry Run

Suppose:

```
k = 3
```

Then:

```
cuts = 0, 1, 2
```

Build `cuts = 0`

Example:

```
dp[1][0][0]
```

Remaining rectangle:

```
1 1 1
0 0 0
```

Contains `1`.

Therefore:

```
dp[1][0][0] = 1
```

Another:

```
dp[2][0][0]
```

Remaining rectangle:

```
0 0 0
```

Contains no 1.

Therefore:

```
dp[2][0][0] = 0
```

---

36. Build `cuts = 1`

Suppose:

```
dp[1][0][1]
```

We need to make one more cut.

Try horizontal cut after row 1.

Given-away:

```
1 1 1
```

Valid.

Remaining state:

```
dp[2][0][0]
```

which is:

```
0
```

Contribution:

```
0
```

Try vertical cut after column `0`.

Given-away:

```
1
0
```

Contains `1`.

Remaining:

```
1 1
0 0
```

State:

```
dp[1][1][0]
```

This contains a `1`.

Therefore:

```
dp[1][1][0] = 1
```

Contribution:

```
1
```

So:

```
dp[1][0][1] = 0 + 1 = 1
```

---

37. Tabulation Code

```
def number_of_ways(matrix, k):

    MOD = 10**9 + 7
```

```
    rows = len(matrix)
    cols = len(matrix[0])

    # ------------------------------------
    # Build prefix sum
    # ------------------------------------

    prefix = [
        [0] * (cols + 1)
        for _ in range(rows + 1)
    ]

    for i in range(1, rows + 1):

        for j in range(1, cols + 1):

            prefix[i][j] = (
                matrix[i - 1][j - 1]
                + prefix[i - 1][j]
                + prefix[i][j - 1]
                - prefix[i - 1][j - 1]
            )

    # ------------------------------------
    # Rectangle sum
    # ------------------------------------

    def rectangle_sum(r1, c1, r2, c2):

        return (
            prefix[r2 + 1][c2 + 1]
            - prefix[r1][c2 + 1]
            - prefix[r2 + 1][c1]
            + prefix[r1][c1]
        )

    # ------------------------------------
    # dp[top][left][cuts]
    # ------------------------------------

    dp = [
        [
            [0] * k
            for _ in range(cols)
        ]
        for _ in range(rows)
    ]

    # ------------------------------------
    # Base case: cuts = 0
    # ------------------------------------

    for top in range(rows):
```

```
        for left in range(cols):

            if rectangle_sum(
                top,
                left,
                rows - 1,
                cols - 1
            ) > 0:

                dp[top][left][0] = 1

    # ----------------------------------
    # Build cuts from 1 to k - 1
    # ----------------------------------

    for cuts in range(1, k):

        for top in range(rows):

            for left in range(cols):

                ans = 0

                # -------------------------
                # Horizontal cuts
                # -------------------------

                for r in range(top, rows - 1):

                    if rectangle_sum(
                        top,
                        left,
                        r,
                        cols - 1
                    ) > 0:

                        ans += dp[
                            r + 1
                        ][
                            left
                        ][
                            cuts - 1
                        ]

                # -------------------------
                # Vertical cuts
                # -------------------------

                for c in range(left, cols - 1):

                    if rectangle_sum(
                        top,
                        left,
```

```
                        rows - 1,
                        c
                    ) > 0:

                        ans += dp[
                            top
                        ][
                            c + 1
                        ][
                            cuts - 1
                        ]

                dp[top][left][cuts] = ans % MOD

    return dp[0][0][k - 1]
```

---

38. Recursion vs Tabulation

Recursion

```
solve(top, left, cuts)
```

asks:

> What is the answer for this state?

Then it recursively asks:

```
solve(smaller_state)
```

Memoization

```
memo[top][left][cuts]
```

stores answers after calculating them.

Tabulation

Instead of asking recursively:

```
Calculate smaller states first.
Then calculate current state.
```

Because:

```
dp[top][left][cuts]
```

depends on:

```
dp[...][...][cuts - 1]
```

we calculate:

```
cuts = 0
    ↓
cuts = 1
    ↓
cuts = 2
    ↓
...
```

---

39. Complete Solution Evolution

```
Step 1:
Understand the cut choices.

        ↓

Step 2:
Define the current remaining rectangle.

        ↓

Step 3:
Represent it using:

(top, left)

        ↓

Step 4:
```

```
Track remaining cuts.

solve(top, left, cuts)

        ↓

Step 5:
Write brute-force recursion.

        ↓

Step 6:
Base case:

cuts == 0

        ↓

Step 7:
Notice repeated states.

        ↓

Step 8:
Memoization.

        ↓

Step 9:
Notice has_one() repeatedly scans rectangles.

        ↓

Step 10:
Build 2D prefix sum.

        ↓

Step 11:
Rectangle contains a 1?

rectangle_sum > 0

        ↓

Step 12:
Convert memoized recursion to tabulation.
```

40. Edge Cases

Case 1: `k = 1`

No cuts are required.

```
k = 1
```

Therefore:

```
cuts = 0
```

The answer is:

```
1
```

if the entire matrix contains at least one `1`.

Otherwise:

```
0
```

Case 2: More pieces than ones

Example:

```
matrix:

1 0
0 0

k = 2
```

Only one `1` exists.

It is impossible for both pieces to contain a `1`.

Answer:

```
0
```

Case 3: Matrix contains no 1

```
0 0
0 0
```

No valid piece can be created.

Answer:

```
0
```

Case 4: k is larger than the number of cells

Impossible.

Each piece must contain at least one 1, so:

```
number of pieces <= number of 1s
```

If:

```
k > number_of_ones
```

answer is:

```
0
```

This is an optional early optimization.

Case 5: A cut gives away an empty piece

Example:

```
0 0 0
---------
1 1 1
```

The top piece contains no 1.

Therefore:

```
Do not recurse.
```

Case 6: Final remaining piece has no `1`

Even if all previous cuts were valid:

```
cuts == 0
```

and the remaining piece has no `1`:

```
return 0
```

---

41. Complexity

Let:

```
R = number of rows
C = number of columns
K = number of pieces
```

Brute Force

The recursion can explore many possible cut sequences.

Additionally, every `has_one()` scans a rectangle.

Very expensive.

Memoization + Prefix Sum

Number of states:

```
R × C × K
```

For each state, we try:

```
O(R) horizontal cuts
+
O(C) vertical cuts
```

Each rectangle check is:

```
O(1)
```

Therefore:

```
Time: O(R × C × K × (R + C))
```

Space:

```
O(R × C × K)
```

for DP.

Prefix sum:

```
O(R × C)
```

additional space.

---

42. Pattern Recognition Template

When you see:

```
2D matrix
+
cut into pieces
+
each piece must satisfy a condition
+
count number of ways
```

Think:

```
1. What is the current remaining region?

2. What are the possible cuts?

3. After a cut, which region remains?

4. What coordinates define that remaining region?

5. What must the discarded piece satisfy?

6. What is the base case when no cuts remain?

7. Does the same state repeat?

8. Can rectangle conditions be answered with prefix sum?
```

---

43. General Recursion Template

```
def solve(state, cuts):

    if cuts == 0:

        return valid(final_state)

    ans = 0

    for every possible cut:

        if discarded_part_is_valid:

            ans += solve(
                remaining_part,
                cuts - 1
            )

    return ans
```

For this problem:

```
def solve(top, left, cuts):

    if cuts == 0:

        return valid(top, left)

    ans = 0
```

```
    for horizontal cut:

        if top_piece_has_one:

            ans += solve(
                r + 1,
                left,
                cuts - 1
            )

    for vertical cut:

        if left_piece_has_one:

            ans += solve(
                top,
                c + 1,
                cuts - 1
            )

    return ans
```

---

44. The Most Important Learning From This Problem

You initially thought:

```
"Hard problem"
   ↓
"DP"
```

But the actual construction was:

```
Current remaining matrix
        ↓
Possible horizontal/vertical cuts
        ↓
Which piece remains?
        ↓
What coordinates describe it?
        ↓
How many cuts remain?
        ↓
Recursion
        ↓
Repeated states
        ↓
```

```
Memoization
        ↓
Expensive rectangle scan
        ↓
2D Prefix Sum
        ↓
Tabulation
```

The crucial mental model is:

```
CUT
 ↓
Give away one part
 ↓
That part must contain a 1
 ↓
Continue with the other part
 ↓
Decrease cuts by 1
```

And the state:

```
solve(top, left, cuts)
```

means:

> **"Starting from this top-left corner, how many ways can I make the remaining number of cuts?"**

That is the complete solution-building journey.